

MLOps Project

Master's in Data Science and Advanced Analytics

NOVA Information Management School

Universidade Nova de Lisboa

Credit Card Default Prediction

An End-to-End, Reproducible MLOps Pipeline

Group: [group name]

Adrian Radoi (20250353)

Lorenzo Simonazzi (20250402)

[third member — name & student number]

Repository: github.com/adriangabrielradoi-art/mlops-credit-default. Run with `uv sync` then `kedro run`, or open `notebooks/00_main.ipynb` and **Run All**.

June 2026

Table of Contents

1. Problem and dataset	3
2. Success metrics	3
3. Project planning (agile sprints)	3
4. Data exploration and modelling results	4
5. Production discussion: advantages, risks, mitigations	5
6. Packages and versions	6

1. Problem and dataset

Objective. Predict whether a credit-card client will **default on next month's payment**, so a lender can act early (adjust limits, intervene, price risk). This is a binary classification task on the UCI *Default of Credit Card Clients* dataset: **29,965 clients** (after de-duplication), 23 raw predictors covering demographics (`LIMIT_BAL`, `SEX`, `EDUCATION`, `MARRIAGE`, `AGE`), six months of repayment status (`PAY_0` to `PAY_6`), bill amounts (`BILL_AMT1` to `BILL_AMT6`) and payment amounts (`PAY_AMT1` to `PAY_AMT6`).

Why this dataset. It is realistic and **imbalanced** (22.1% defaults), it has a clear business owner and decision, and it supports a credible **drift** story (an applicant pool can age or shift demographically over time), which fits an MLOps course well. It is public, well-sized for a free-tier feature store, and every feature is interpretable, which makes the SHAP section meaningful.

Target. `default` (1 = default next month, 0 = not), prevalence **22.1%**.

2. Success metrics

The classes are imbalanced, so plain accuracy is misleading (predicting “never default” scores about 78%). We defined success up front as:

- **Primary: ROC-AUC ≥ 0.80** on a held-out test set. Threshold-independent, stable under class imbalance, and the standard for credit scorecards.
- **Secondary: PR-AUC and F1.** PR-AUC focuses on the positive (default) class we actually care about; F1 balances precision and recall at the decision threshold.
- **Operational:** the model loads at service startup (no rebuild to promote a model), single-prediction latency suitable for online use, and a drift detector that fires on a shifted batch while staying quiet on a clean one.

Outcome: the champion reaches **ROC-AUC 0.801** on the hold-out test set, so the primary target is met.

3. Project planning (agile sprints)

We organised the work as one **Kedro pipeline per concern** (eight in total), each an ownership boundary, and scheduled them over five one-week sprints to the 30/06 deadline.

Sprint	Focus (pipelines)	Lead
1	Group setup, dataset choice, EDA, <code>data_quality</code> (Great Expectations)	Adrian
2	<code>data_cleaning</code> , <code>data_feat_engineering</code> + Hopsworks feature store	Lorenzo
3	<code>data_split</code> (+ drift reference), <code>model_train</code> (MLflow + Optuna)	[member 3]
4	<code>model_selection</code> (+ SHAP, registry), <code>model_predict</code> (FastAPI + Docker)	Adrian / Lorenzo
5	<code>data_drifts</code> (Evidently), unit tests, report, packaging	all

Cross-cutting infrastructure (MLflow tracking, the `data/` 8-layer convention, the catalog and parameters) was set up in Sprint 1 so every later pipeline plugged into it. We kept **exploration notebooks** (`notebooks/00` to `08`) alongside the modular code: one companion notebook per pipeline, plus `00_main.ipynb` as a one-click control panel.

4. Data exploration and modelling results

EDA and cleaning decisions. The raw data contained **undocumented category codes** the data dictionary does not define: `EDUCATION` in `{0,5,6}` and `MARRIAGE` = `0`. Rather than drop these rows (losing data) or leave them (polluting the signal), we **merged them into the existing “other” bucket** (`EDUCATION` 0/5/6 to 4, `MARRIAGE` 0 to 3). This was a deliberate, documented cleaning choice recorded in `notebooks/02_data_cleaning.ipynb`. We **kept** the `PAY_*` = `-2/-1` codes (no consumption or paid in full): they are about 22% of rows and carry real repayment signal. After de-duplication: **29,965 rows, 24 columns**.

Feature engineering. From the six-month history we derived behavioural features that compress the time series into model- and SHAP-friendly signals: worst and mean repayment delay (`pay_delay_max`, `pay_delay_mean`), months in arrears (`n_months_delayed`), average and max bill, bill trend, total and average payment, **credit utilisation** (`util_ratio`) and **repayment ratio** (`pct_paid`). The table grew from 24 to **36 columns**. These features were registered as four **Hopsworks feature groups** (demographic, payment-status, bill, payment-amount), keyed by `customer_id`.

Model comparison. We tuned three families with **Optuna** (15 trials each), selecting on validation ROC-AUC, every run tracked in **MLflow**:

Model	ROC-AUC	PR-AUC	F1	Accuracy
Gradient Boosting (champion)	0.801	0.595	0.480	0.825
Random Forest	0.799	0.599	0.556	0.776
Logistic Regression	0.778	0.542	0.546	0.762

Gradient Boosting wins on ROC-AUC (our primary metric) and is **registered in the MLflow Model Registry** under the alias `Champion`. Random Forest is within 0.002 and has a better F1 and PR-AUC, a genuine trade-off. If recall on defaulters mattered more than ranking quality, RF (or a lower decision threshold) would be defensible.

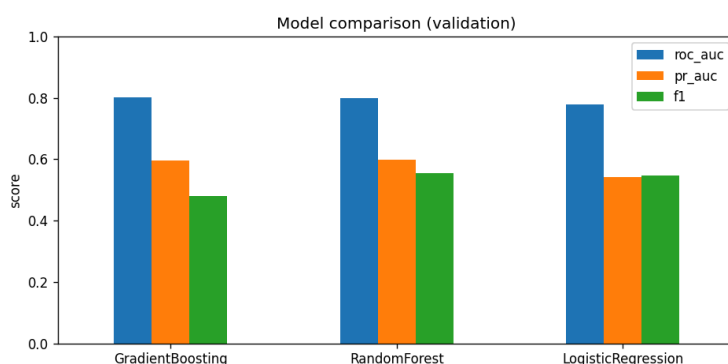


Figure 1. Validation ROC-AUC across the three Optuna-tuned model families.

Explainability (SHAP). The global SHAP summary (`shap_summary.png`) ranks the most recent repayment status `PAY_0` first, followed by **our engineered features** `pay_delay_max`, `n_months_delayed`, `pay_delay_mean`, then `util_ratio` and `LIMIT_BAL`. This is both **intuitive** (recent repayment behaviour dominates credit risk) and a **validation of the feature engineering**: four of the top features are ones we built. Per-prediction SHAP is logged for scored clients so any decision can be explained and audited.

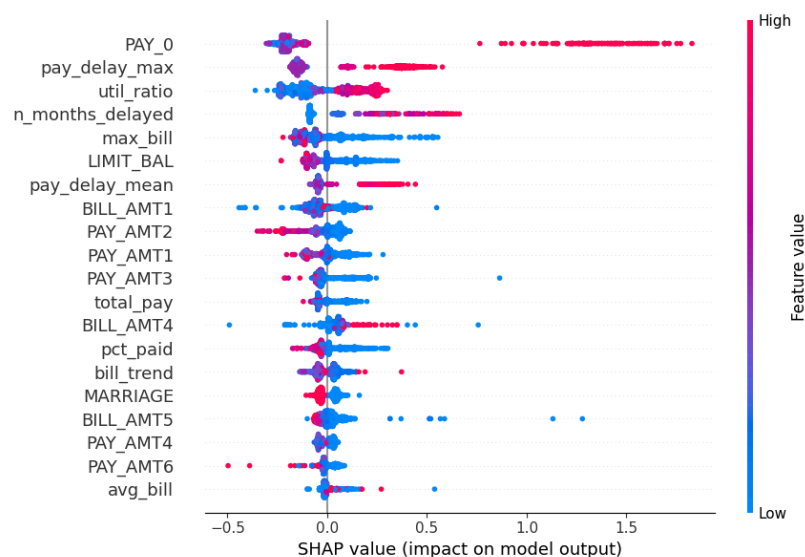


Figure 2. Global SHAP summary: repayment-status and engineered delay and utilisation features dominate.

Drift validation. To prove the monitoring works on a static dataset, we compared the training reference against two test batches: a **clean** control and a **biased** batch (older clients, AGE $\geq$ 85th pct). The detector behaved exactly as intended: **0 features flagged on the clean batch, 3 on the biased batch** (AGE, MARRIAGE, EDUCATION, the demographics that move with age; PSI for AGE = 7.7). The **multivariate PCA reconstruction stayed stable**: this is a *marginal* demographic shift that univariate PSI catches but that does not break the feature correlation structure. The two methods are complementary by design. Batch inference scored the **5,993** test clients at a 11.8% predicted default rate.

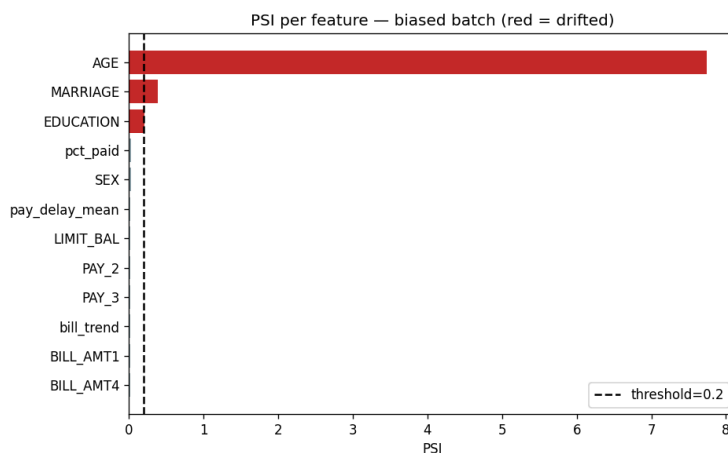


Figure 3. PSI per feature on the biased batch. Only the age-correlated demographics cross the 0.20 drift threshold (red).

5. Production discussion: advantages, risks, mitigations

This is a proof of concept. Below is how each technology would behave in production and what we would do about its limits.

Technology	Production advantage	Risk	Mitigation
Kedro	Modular, testable, run full DAG or any stage alone	Not a scheduler by itself	Wrap pipelines in Prefect /Airflow for scheduling and retries
MLflow	Experiment tracking + model registry (alias-based promotion)	Local sqlite store does not scale to a team	Move to a remote MLflow server + Postgres/artifact store
Great Expectations	Fail-loud data unit tests, gate halts on bad data	Suites can drift from reality	Version suites; review on schema change
Hopsworks	Central feature store, online lookups for serving	Free serverless tier: offline (HDFS) writes blocked from an external client	We wrote to the online store (RonDB); in production a managed Spark cluster materialises the offline store for training
FastAPI + Docker	Low-latency online serving, reproducible runtime, model loaded at startup (no rebuild to promote)	One process per container; single host	Horizontal scaling on Kubernetes behind a load balancer
SHAP / Evidently	Per-prediction explanations + drift reports	SHAP cost grows with traffic	Sample requests for explanation logging
pandas	Easy to write, fine at this scale	Single-machine only	Re-implement in Spark/Polars if data exceeds about 50M rows

The Hopsworks story (a real risk we hit and mitigated). Writing the offline feature store from our machines failed with an HDFS `RPC listener disconnected` error. The free serverless tier does not accept Delta-Lake/HDFS writes from an external Python client. We diagnosed this, switched the feature groups to **online storage (RonDB)**, and the write and round-trip read of **29,965 rows across 4 groups** then succeeded. In a real deployment with managed Spark compute, the offline store materialises normally; the online store is what serving reads anyway.

Deployment strategy. To promote a new champion we recommend **canary** releases: route a small percentage of traffic to the challenger and scale up only if ROC-AUC and PR-AUC hold, because a credit decision has real customer impact and we want a fast, observable rollback. **Shadow** mode is a good pre-step (log challenger predictions without using them); **blue/green** is simpler but exposes all traffic at once. The MLflow alias makes rollback a one-line re-point.

Scaling roadmap. Single Docker today, then Docker Compose (API plus MLflow), then **Kubernetes with horizontal pod autoscaling** when latency SLAs are breached at peak, and finally **Kubeflow** if we operate several production pipelines or models.

Monitoring and retraining playbook. (1) log every decision (`decision_log.json`, `predictions.parquet`); (2) compute drift per batch (PSI/JS/KS plus PCA, `drift_summary`); (3) adjust the decision threshold before retraining (`prediction_threshold`); (4) trigger a retrain on sustained drift (`trigger_retrain_on_drift`); (5) escalate to new features or shadow A-B testing for v2.

Quality engineering. 19 unit tests cover the pipeline nodes and the serving API; the full DAG runs end-to-end and is reproducible (fixed seeds, `uv.lock`, MLflow run IDs as the join key between experiments and artifacts).

6. Packages and versions

Direct dependencies (Python 3.13, `.venv-mlops`); full transitive pins in `uv.lock`.

Role	Packages (version)
Orchestration	kedro 1.3.1, kedro-datasets 9.4.0, kedro-mlflow 2.0.2
Experimentation	mlflow 3.11.1, optuna 4.8.0
Data quality / feature store	great-expectations 1.16.1, hopsworks 5.0.0
Modelling	scikit-learn 1.8.0, shap 0.52.0, pandas 2.3.3, pyarrow 23.0.1, xlrtd 2.0.2
Serving	fastapi 0.136.0, uvicorn 0.44.0, docker 7.1.0
Monitoring	evidently 0.7.21
Scheduling	prefect 3.6.27
Tooling	ucimlrepo 0.0.7, jupyterlab 4.5.6, ipywidgets 8.1.8
Testing	pytest 9.1.1, pytest-cov 7.1.0